Multi-threading : mutual exclusion, atomics, and independence

Block 1. Mutual Exclusion

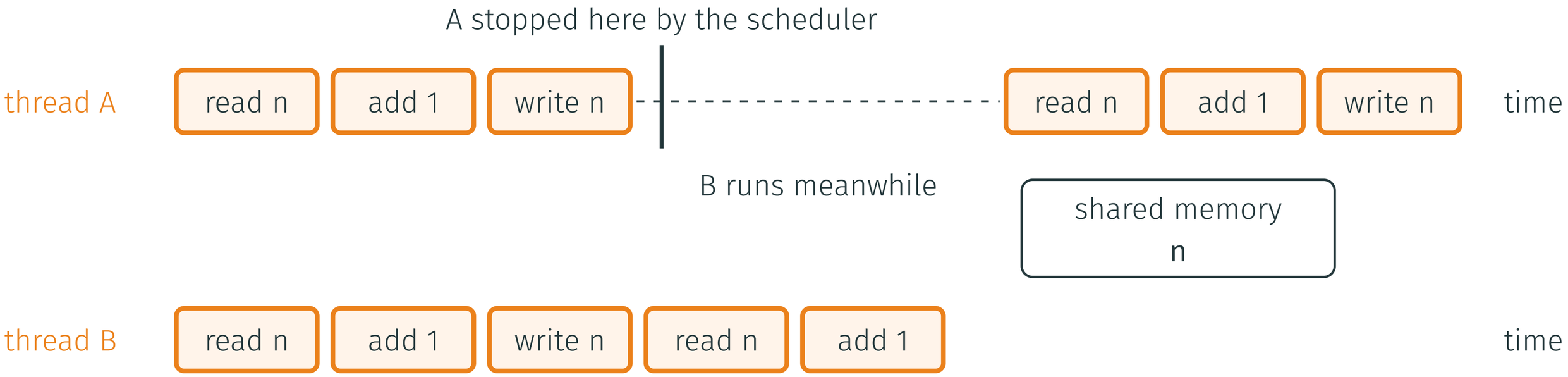
Which ways control the interleavings of a multi-threaded program, and what does each guarantee, require and risk?

PART 1

Threads and shared memory

the scheduler, interleavings, sequential consistency

Threads, memory and the scheduler



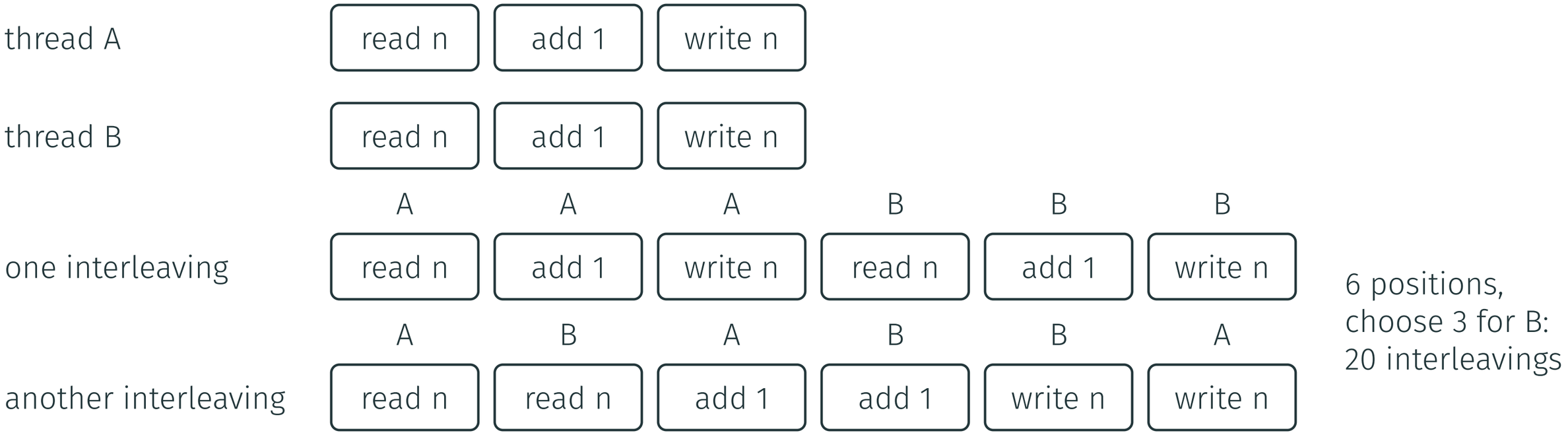
Instruction. One step the processor executes as a whole: a read of one memory location into the processor, a write of one value to one location, an operation inside the processor.

Shared memory. Locations that every thread can read and write by their address: fields and array elements, not the local variables of a Java method.

Scheduler. Decides which thread runs where; may stop a thread between any two instructions and resume it later.

Its parts: a queue of the threads ready to run; a policy that picks the next one; a timer interrupt that stops the running thread; a context switch that saves its registers and program counter and loads another's.

The interleaving



6 positions,
choose 3 for B:
20 interleavings

> **Definition.** An **execution** of a multi-threaded program is an **interleaving** of its threads' instructions: one sequence that contains every instruction of every thread, each thread's instructions in the order the thread issues them.

Two threads that each execute a million three-step increments have more interleavings than any test could run.

> **! A correctness argument does not look at runs. It should cover every interleaving at once; that is what the rest of this block does.**

Four assumptions

1. **Several threads.** Two or more, each a sequence of instructions.

2. **One shared memory.** All threads read and write the same locations.

3. **A scheduler that may stop a thread between any two instructions.** So an execution is an interleaving, and any interleaving may occur.

4. **Sequential consistency.** There is one memory, and every read returns the value of the latest write to the same location before it in the interleaving; with assumption 3, this is Lamport's sequential consistency*.

   Each processor has a cache and compilers reorder, so this is not literally what the hardware does: a guarantee of the language, given only for programs without **data races**, defined in part 2.

* L. Lamport, IEEE Transactions on Computers C-28(9), September 1979, pages 690–691: the definition.

PART 2

Race conditions

data races and atomicity violations

Example 1: the counter, unprotected

Java | OCaml 5

CounterRacy.java ↗

```
static int n = 0;   // shared

Runnable work = () -> {
  for (int i = 0; i < N; i++) n++;
};
Thread t1 = new Thread(work),
       t2 = new Thread(work);
t1.start(); t2.start();
t1.join();  t2.join();
System.out.println("got " + n);
```

Two threads each add one to `n`, N times. What does the program print as `n`?

1. Always $2N$.
2. **At most $2N$.**
3. Possibly more than $2N$.
4. Always less than $2N$.

```
$ java CounterRacy
expected 2000000
got      1017204
```

One sample of a run. Ten further runs printed ten other numbers, none of them 2,000,000.

The three steps of n++

Java | OCaml 5

CounterRacy.java ↗

```
int v = n;      // read
v = v + 1;      // add
n = v;          // write
```

The Java compiler produces four bytecodes, `getstatic`, `iconst_1`, `iadd`, `putstatic`: the first is the read, the last the write, and the scheduler may run another thread between them.

> **Lost update.** A write whose effect is overwritten by a write based on an older read.

Step	Thread A	Thread B	n
1	read n, gets 5		5
2		read n, gets 5	5
3	add: register 6		5
4	write 6		6
5		add: register 6	6
6		write 6	6

The table is the interleaving that loses an update, from `n` equal to 5: both read 5, both write 6; "register" is the thread's own copy of the value. Two increments were executed and `n` grew by one.

The interleavings of two increments

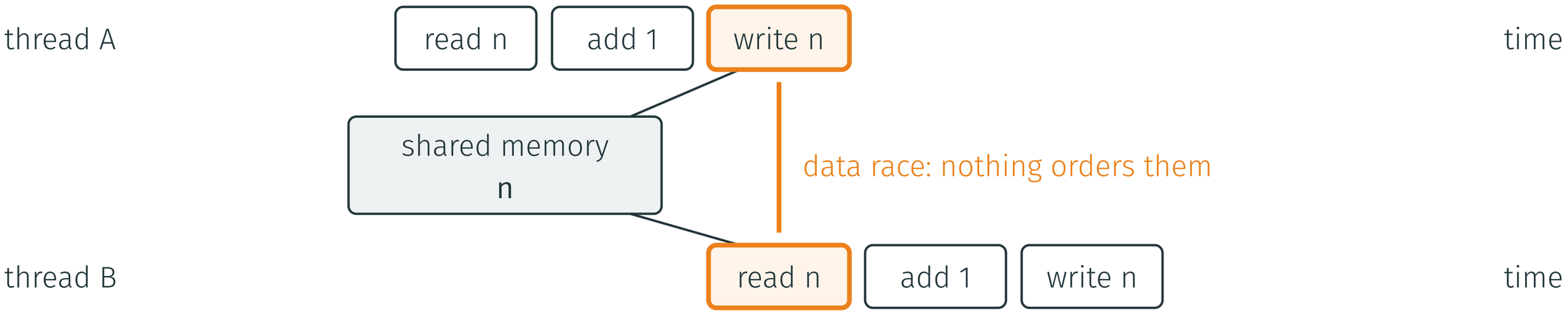
`r1 a1 w1 r2 a2 w2`	~~`r1 a1 r2 w1 a2 w2`~~	~~`r1 a1 r2 a2 w1 w2`~~	~~`r1 a1 r2 a2 w2 w1`~~
~~`r1 r2 a1 w1 a2 w2`~~	~~`r1 r2 a1 a2 w1 w2`~~	~~`r1 r2 a1 a2 w2 w1`~~	~~`r1 r2 a2 a1 w1 w2`~~
~~`r1 r2 a2 a1 w2 w1`~~	~~`r1 r2 a2 w2 a1 w1`~~	~~`r2 r1 a1 w1 a2 w2`~~	~~`r2 r1 a1 a2 w1 w2`~~
~~`r2 r1 a1 a2 w2 w1`~~	~~`r2 r1 a2 a1 w1 w2`~~	~~`r2 r1 a2 a1 w2 w1`~~	~~`r2 r1 a2 w2 a1 w1`~~
~~`r2 a2 r1 a1 w1 w2`~~	~~`r2 a2 r1 a1 w2 w1`~~	~~`r2 a2 r1 w2 a1 w1`~~	`r2 a2 w2 r1 a1 w1`

The 720 orders of the six steps `r1 a1 w1 r2 a2 w2`, one per cell, in lexicographic order. In orange, the 20 that keep each thread's own order, `r` before `a` before `w`: the interleavings.

Of the 720 orders of the six steps, program order within each thread leaves these 20.

18 of the 20 read before the other thread's write and lose an update; 2 count correctly: one increment after the other.

Data race

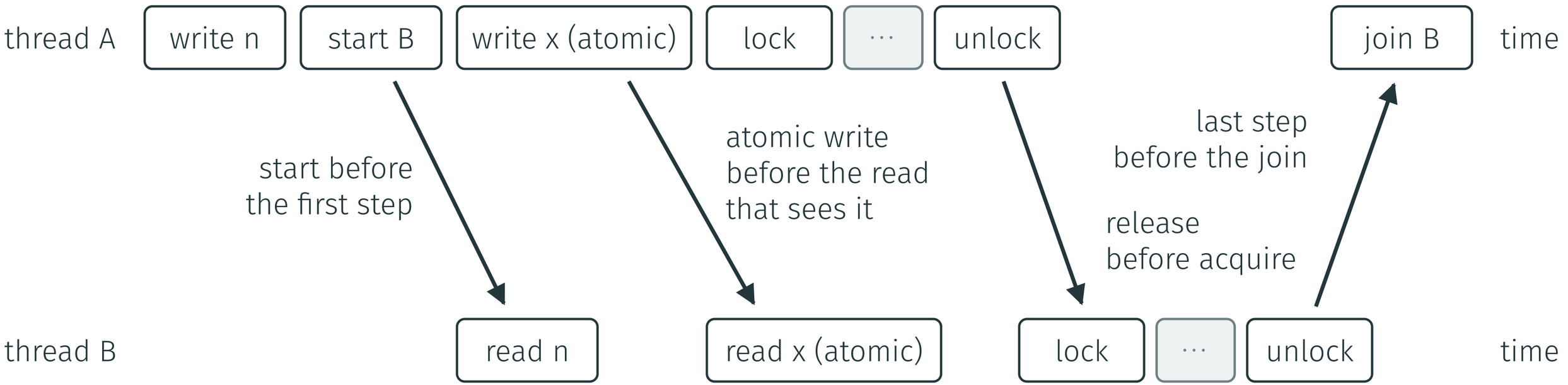


> **Data race.** Two accesses to one memory location from two threads, at least one a write, with nothing of the program ordering one before the other.*

> To avoid the race, the two threads must coordinate: what can the program put between the two accesses?

* The Java Language Specification, chapter 17, section 17.4.5; C++, Go and Rust define a data race the same way.

Synchronization mechanisms



A lock. Its release is ordered before the acquire that follows it: what the releasing thread wrote before, the thread that acquires next sees after.

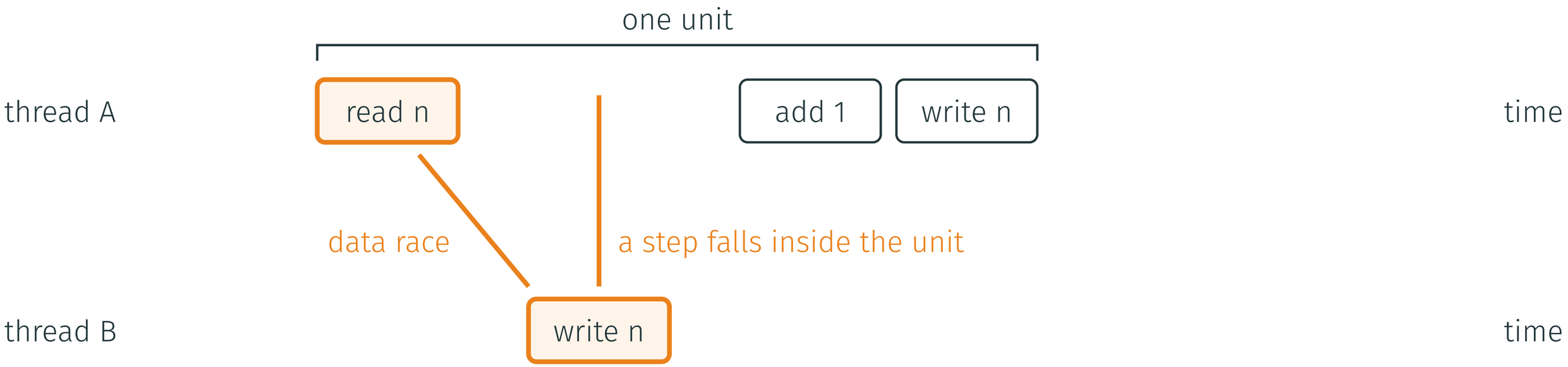
A thread's start and join. The call of `start` is ordered before the new thread's first instruction, and its last instruction before the return of the `join` that waits for it.

An atomic. A write to an atomic or volatile variable is ordered before the read that sees it, with everything the writer did before.

Each mechanism orders one thread's instructions before another's: what the first thread wrote before, the second sees after. The counter has none of them between its accesses, and that is its data race.

But is a data race the only fault in the counter?

Atomicity violation



> **Atomicity violation.** An operation of several steps meant as a unit, **atomic**, every other thread seeing all of it or none, and not protected as a unit, so that another thread's step can fall between two of its steps.

The counter is both.

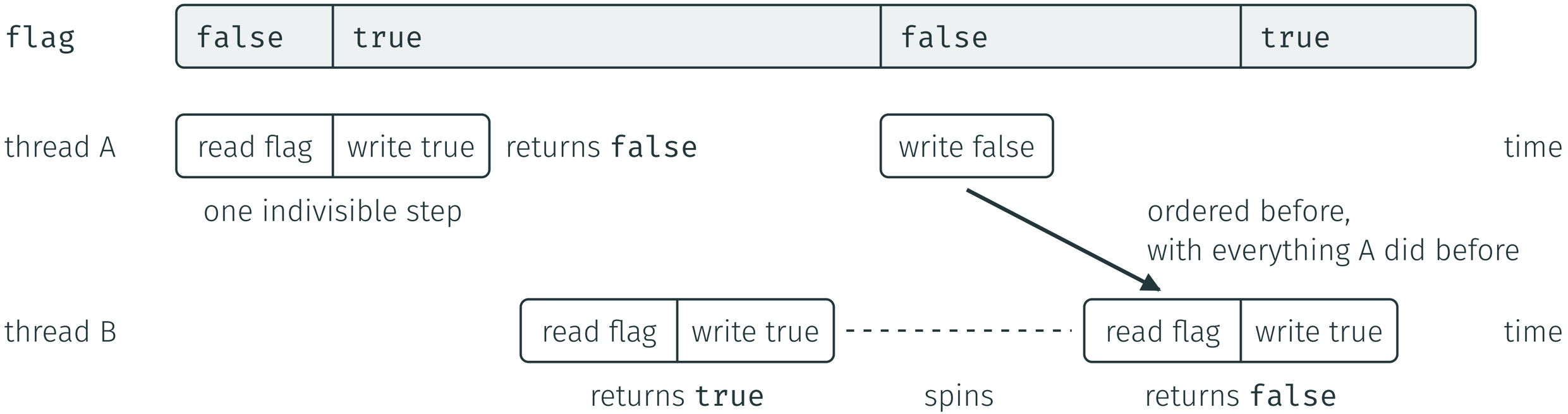
In part 6's transfer each access is under its lock: no data race, and still an atomicity violation.

PART 3

Atomic instructions

test-and-set and spin locks

Test-and-set and its guarantees



Indivisibility. It reads the old value and writes the new one as one step: no thread sees the location between.

Synchronization. The write of `false` that releases the flag is ordered before the test-and-set that reads it, and with it everything the releasing thread did before.

Example 2: the spin lock

Java | OCaml 5

SpinLock.java ↗

```
static final AtomicBoolean held =
    new AtomicBoolean(false);
static void lock() {
  while (held.getAndSet(true)) { }
}
static void unlock() { held.set(false); }

static int n = 0;   // plain, as in example 1
// the loop of each of the two threads
for (int i = 0; i < N; i++) {
  lock(); n++; unlock();
}
```

Java provides the instruction as `getAndSet(true)`, a method with the two guarantees, carried out by the processor's own atomic instruction where it has one; OCaml 5 as `Atomic.exchange flag true`.

```
$ java SpinLock
expected 2000000
got      2000000
```

One sample of a run.

The lock is the instruction with a loop around it. The counter is exact for two reasons, the two guarantees: at most one thread at a time is between `lock()` and `unlock()`, and the release orders its write of `n` before the next holder's read.

PART 4

Locks

critical sections; contract, obligation and risks

Example 3: the counter under one lock

Java | OCaml 5

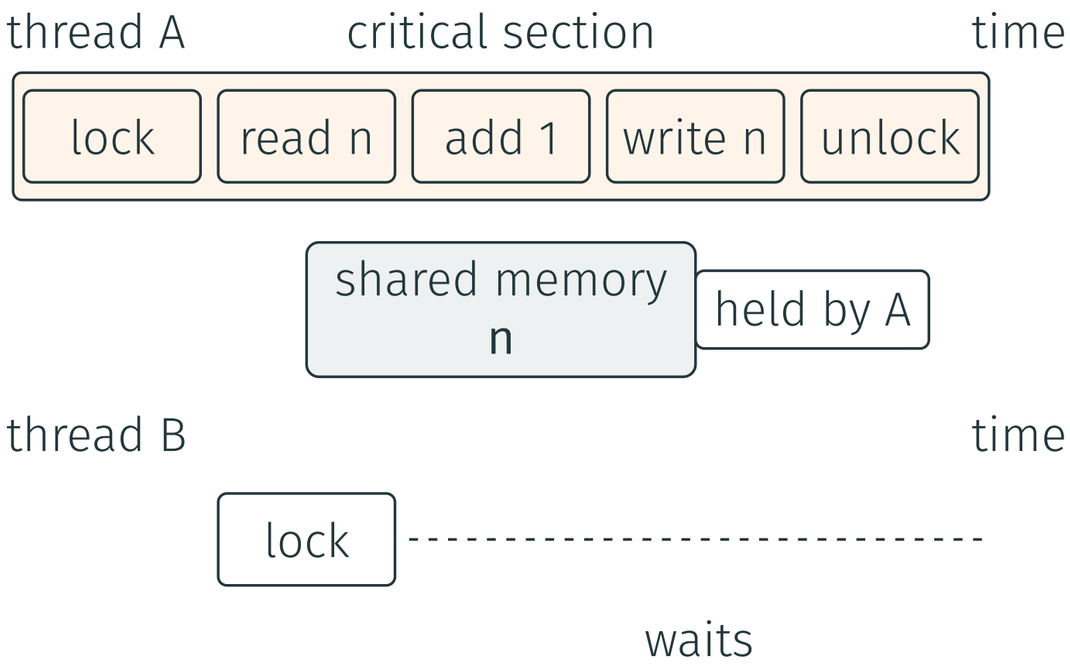
CounterLocked.java ↗

```java
static final Object lock = new Object();
static int n = 0;   // guarded by lock

// the loop of each of the two threads
for (int i = 0; i < N; i++) {
  synchronized (lock) {   // critical
    n++;                  // section
  }
}
```

```
$ java CounterLocked
expected 2000000
got      2000000
```

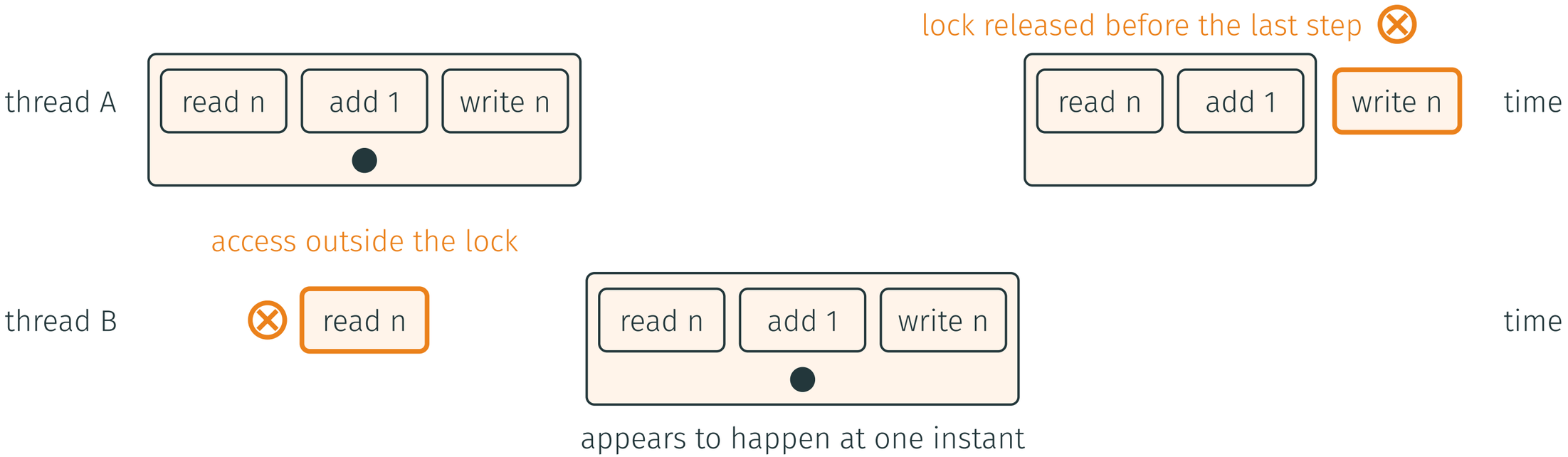
A waiting thread is **suspended** until the release, not spinning; the closing brace releases the lock even on an exception.



Lock. An object with two operations, **acquire** and **release**: between a thread's acquire that returned and its release, no other thread's acquire returns.

Critical section. The instructions a thread executes while holding the lock.

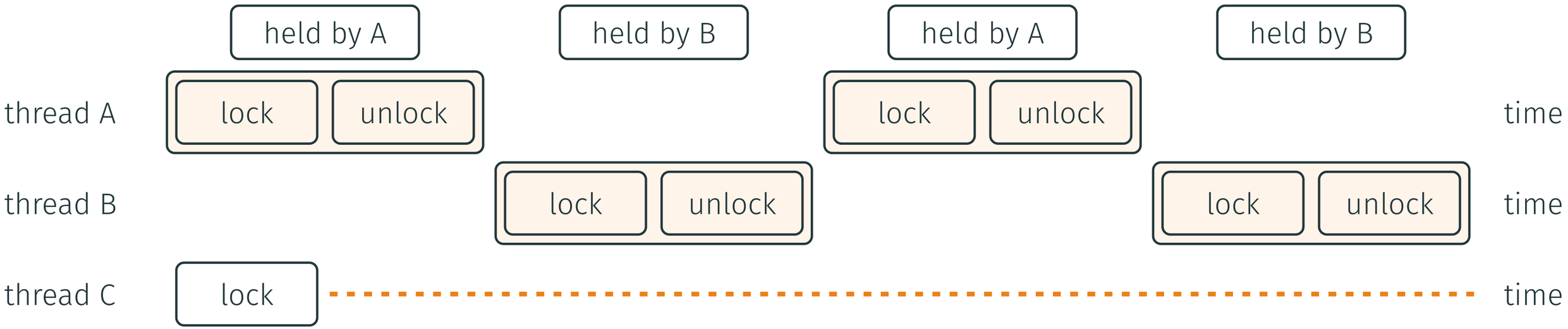
The contract and the obligation



Contract. Operations are atomic by exclusion: under the lock they happen one after another, and each appears to the other threads to happen at one instant inside its critical section.

Obligation, two parts here. Every access to the protected state, read or write, while holding the lock; and the lock held for the whole operation, over every piece of state its **invariant** mentions, the statement that holds whenever no thread is inside a critical section.

The progress risk of a lock



Progress risk, first source: a stopped holder. A thread that holds a lock and is stopped stops every thread that needs it. Stopped in a cycle, it is deadlock; not scheduled, a missed deadline, in the appendix.

Second source: starvation. A lock that keeps changing hands may never reach one thread. An order among the waiters removes it, and block 2's wait-freedom rules it out.

And who checks that every access is under the lock -- programmer's discipline or can a language enforce it ?

PART 5

Race detection and prevention

run-time detectors; ownership in Rust

Example 4d: the unlocked reader

Java | OCaml 5

Accounts.java ↗

```
// (c) both locks, in the order of the ids
static void transferC(Account from,
                      Account to, int x) {
  Account first  =
      from.id < to.id ? from : to;
  Account second =
      from.id < to.id ? to : from;
  synchronized (first) {
    synchronized (second) {
      from.balance -= x; to.balance += x;
    }
  }
}
// (d) a reader that takes no lock
static int[] readUnlocked() {
  return new int[] { A.balance, B.balance };
}
```

Two accounts, A (id 1) and B (id 2), one lock each; t1 and t2 transfer in opposite directions; a reader takes no lock. Does it compile, and what did the compiler establish about the reader's reads?

1. No: a balance is read outside its lock.
2. Yes; the runtime reports the unlocked read.
3. **Yes; the two reads are data races, and nothing reports them.**
4. Yes; reading an `int` is one instruction, so the reads are correct.

```
$ java Accounts d
readings with no lock: 138670249
sums other than 200:    20954124
first wrong: A = -5811, B = 6012
```

Who checks the obligation

Language	Who checks that every access is under the lock
Java, OCaml 5	Nobody. A comment beside the field, `// guarded by lock`, is the only record of the rule.
Go; C and C++	Nobody by default; a detector at run time when run with Go's `-race` or ThreadSanitizer, a race detector in clang and gcc: it reports two unordered accesses the run made, even when the timing kept them apart and the answer was right, as far as its record of recent accesses reaches; a path the run did not take is not checked.
Rust	The compiler, before any run, for data races only.

For one word, a variable of one location, that one thread writes and another reads, Java offers `volatile`: every write is synchronization for the read that sees it. `n++` on a `volatile` field is the same three steps as before.

Rust: the counter rejected

Rust

r1a_scoped_mut.rs ↗

```rust
let mut count = 0;
thread::scope(|s| {
  s.spawn(|| for _ in 0..1_000_000 {
    count += 1
  });
  s.spawn(|| for _ in 0..1_000_000 {
    count += 1
  });
});
println!("{}", count);
```

```
error[E0499]: cannot borrow `count` as mutable
                more than once at a time
  |
6 |   s.spawn(|| for _ in 0..1_000_000 { count += 1 });
  |   -------    first mutable borrow occurs here
7 |   s.spawn(|| for _ in 0..1_000_000 { count += 1 });
  |           ^^ second mutable borrow occurs here

error: aborting due to 1 previous error
```

Ownership, the intuition: every value has one owner at a time, and a value may be shared by several threads or changed by one, never both at once, unless a lock or an atomic stands between*. A **mutable borrow** is a reference through which a thread may change a variable; two threads that both change `count` are two mutable borrows, and the compiler rejects the second.

* Session 12 gives the rule.

Rust: `Mutex<T>`, the lock and its data

Rust

r1b_arc_mutex.rs ↗

```rust
let count = Arc::new(Mutex::new(0));
let mut handles = Vec::new();
for _ in 0..2 {
    let c = Arc::clone(&count);
    handles.push(thread::spawn(move || {
        for _ in 0..1_000_000 {
            *c.lock().unwrap() += 1;
        }
    }));
}
for h in handles { h.join().unwrap(); }
println!("{}", *count.lock().unwrap());
```

```
$ ./r1b_arc_mutex
2000000
```

A `Mutex<T>` holds the `T`; `Arc` lets the threads share the lock. While it is shared, the only way to the `T` is the **guard** that `lock()` returns, which releases the lock when it goes out of scope: no thread reaches the integer without holding its lock.

> Rust checks the first part of the obligation, every access under its lock; the others remain the programmer's.

> Not checked: the order of the locks, so the deadlock of part 6 compiles; nor atomicity, so the transfer in two halves compiles. A compiler can rule out a data race, not an atomicity violation.

PART 6

Deadlock

lock ordering; safety and liveness

Example 4a: the transfer in two halves

Java | OCaml 5

Accounts.java ↗

```
// (a) each step under its lock,
//     the transfer as a whole under none
static void transferA(Account from,
                      Account to, int x) {
  synchronized (from) { from.balance -= x; }
  synchronized (to)   { to.balance   += x; }
}
// the reader: both balances, both locks
static int[] read() {
  synchronized (A) { synchronized (B) {
    return new int[] { A.balance, B.balance };
  } }
}
```

A and B hold 100 each; the invariant is that the sum is 200. The withdrawal and the deposit are each under a lock. Is the transfer atomic?

1. Yes: each of its two steps is atomic, so the sequence of the two is.
2. **No: between the two steps a thread holding both locks can read a sum of 199.**
3. No, and the reads of the balances are data races.
4. Yes, provided the reader takes both locks.

```
$ java Accounts a
reading 1: A = 99, B = 100,
           sum = 199
```

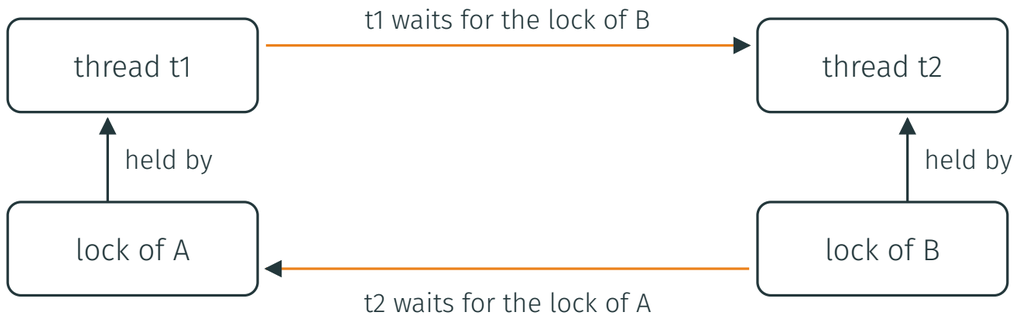
Example 4b: both locks, deadlock

Java | OCaml 5

Accounts.java ↗

```java
// (b) both locks, the source's first
static void transferB(Account from,
                      Account to, int x) {
  synchronized (from) {
    synchronized (to) {
      from.balance -= x; to.balance += x;
    }
  }
}
ThreadMXBean bean =
    ManagementFactory.getThreadMXBean();
while (bean.findDeadlockedThreads() == null)
  Thread.sleep(50);
```

```
$ java Accounts b
deadlock: 2 threads blocked for ever
  t1 holds account A,
     waits for account B, held by t2
  t2 holds account B,
     waits for account A, held by t1
```



> **Deadlock.** A state in which every thread of a set waits for a lock held by another thread of the set, so that none of them can ever proceed.

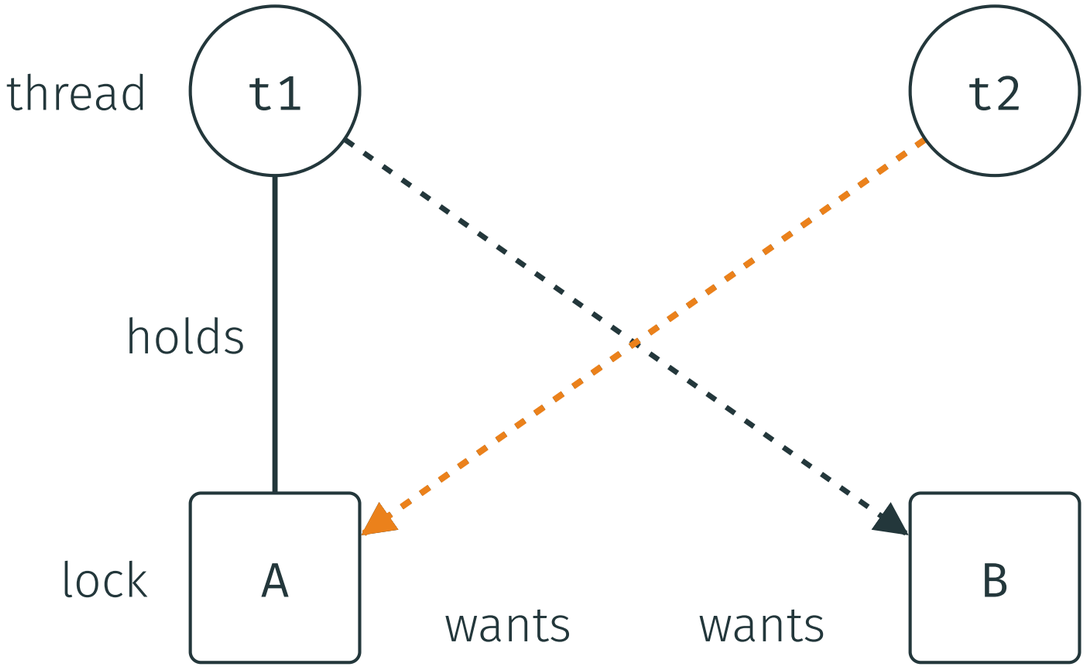
Safety and liveness

> **Safety property.** Something bad never happens; a violation is a finite run, one that reaches the bad state. **Liveness property.** Something good eventually happens; no finite run can show a violation, since the good thing may still come.*

The deadlock violates both kinds. That each thread holds one lock and waits for the other's is a state, and the detector finds it: "never deadlocked" is a safety property. That no transfer will ever complete is the liveness property it ruins; no detector can see "ever".

* Lamport 1977 (IEEE TSE SE-3(2), 125–143); Alpern and Schneider 1985 (Inf. Proc. Letters 21(4), 181–185).

The Coffman conditions (1971)



both take A first: no cycle

Four conditions are all necessary for a deadlock:

1. **Mutual exclusion.** A resource is held by one thread at a time.
2. **Hold and wait.** A thread holds a resource while it waits for another.
3. **No preemption.** A held resource cannot be taken away.
4. **Circular wait.** A cycle of threads, each waiting for a resource the next holds.

This session removes the fourth.

Example 4c: a global order of the locks

Java | OCaml 5

Accounts.java ↗

```java
// (c) the locks always taken
//     in the order of the ids
static void transferC(Account from,
                      Account to, int x) {
  Account first  =
      from.id < to.id ? from : to;
  Account second =
      from.id < to.id ? to : from;
  synchronized (first) {
    synchronized (second) {
      from.balance -= x; to.balance += x;
    }
  }
}
```

```
$ java Accounts c
transfers: 1000000 A to B,
           1000000 B to A
readings: 530369, wrong sums: 0
at the end: A = 100, B = 100
```

Every thread that takes several locks takes them in the **same order**. A thread waits only for a lock **later in the order** than every lock it holds; along a chain of waiting threads the locks come later and later, and a chain that only goes up **cannot return to its start**.

The order is the **third part of the obligation**. **Nobody checks it** in Java, OCaml, Go or C, and the deadlock **compiles in safe Rust**.

Summary

mutual exclusion: guarantee, obligation, progress risk

Mutual exclusion in summary

The guarantee	Operations atomic by exclusion: under the lock they happen one after another, each at one instant inside its critical section.
The obligation	Every access to the protected state under the lock; every lock the invariant needs, held for the whole operation; one global order of the locks.
The progress risk	A stopped holder stops every thread that needs its lock: deadlock when the holders wait in a cycle, a missed deadline when the holder is not scheduled. And starvation, when the lock keeps changing hands but one thread is passed over for ever.
Who checks	Nobody by default, in Java, OCaml 5, Go, C and C++; a detector at run time, with Go's `-race` or ThreadSanitizer, for the accesses a run made; Rust's compiler, for data races only, never the lock order or atomicity.

The question for block 2



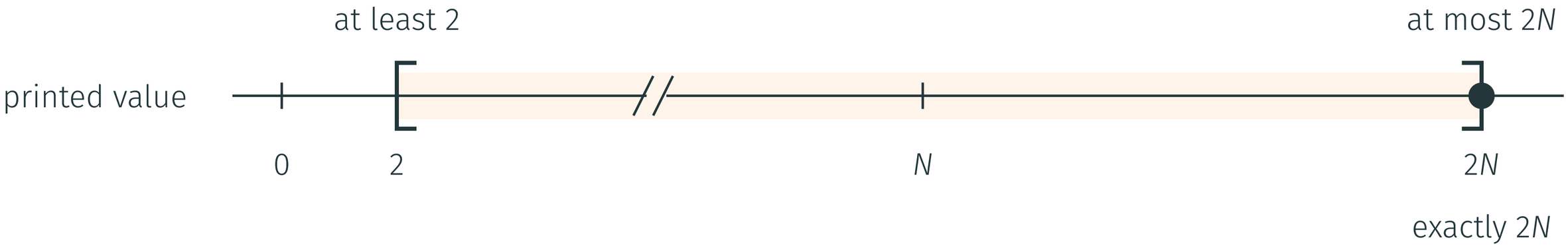
> Then can an operation be atomic without a lock around it?

Block 2 answers three questions this block leaves open: what an atomic operation without a lock guarantees and requires; how to prove such an operation correct; and, at its end, what the language guaranteed about memory in the fourth assumption, and what holds instead for a program with a data race.

Bounds and priority inversion

the bounds on the printed value; Mars Pathfinder and priority inheritance

The bounds on the printed value

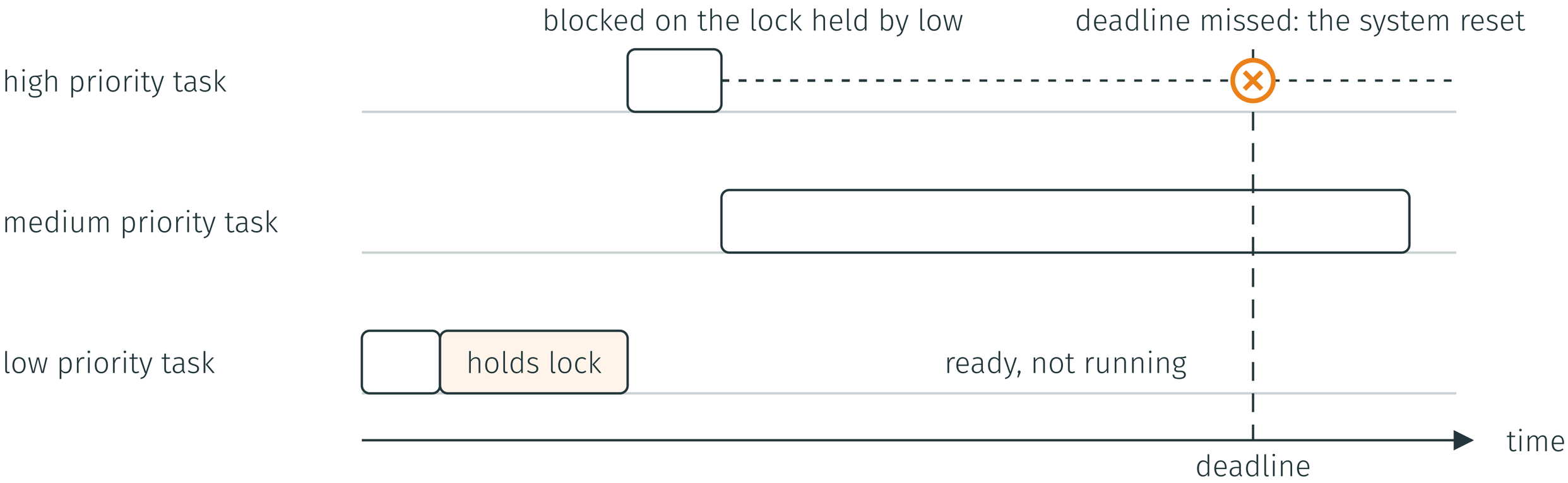


At most 2*N*: every write stores one more than a value read.

Exactly 2*N* is possible: in every interleaving in which no two increments overlap, that is, no read of `n` falls between another increment's read and its write.

At least 2, for every *N* of at least two.

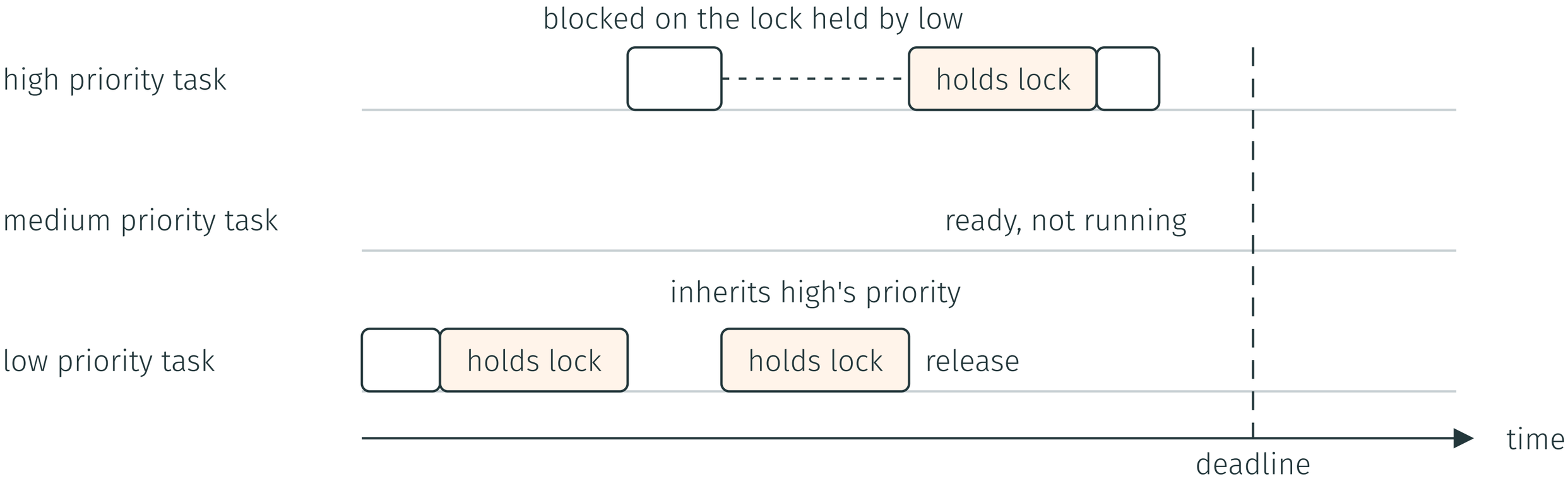
Mars Pathfinder (1997)



The cause was a **priority inversion**: a high-priority task waited for a lock held by a low-priority task, which medium-priority tasks kept from running.

The lock did its job: one task at a time held it. What failed was progress: the high-priority task missed a hard deadline, and the system reset. Not a deadlock: one lock, no cycle. An unbounded **priority inversion**: the holder was ready to run, and the scheduler kept choosing the medium tasks.

Priority inheritance and the priority ceiling



The remedy was **priority inheritance**: a task holding a lock runs at the priority of the highest task waiting for it. JPL turned it on by a patch sent from Earth.

> Inheritance alone does not prevent deadlock on two locks taken in opposite order; a fixed order does, anywhere, and on one processor so does the priority ceiling protocol (Sha, Rajkumar and Lehoczky, 1990).

APPENDIX

Condition synchronization

semaphores, monitors and condition variables

Dijkstra's bounded buffer (1965)

> **Semaphore.** A counter with two indivisible operations: **P** waits until the counter is above zero and then decreases it by one; **V** increases it by one. In the Part 2 introduction and in Java's library: `acquire` and `release`.*

Three semaphores around a queue of *N* places:

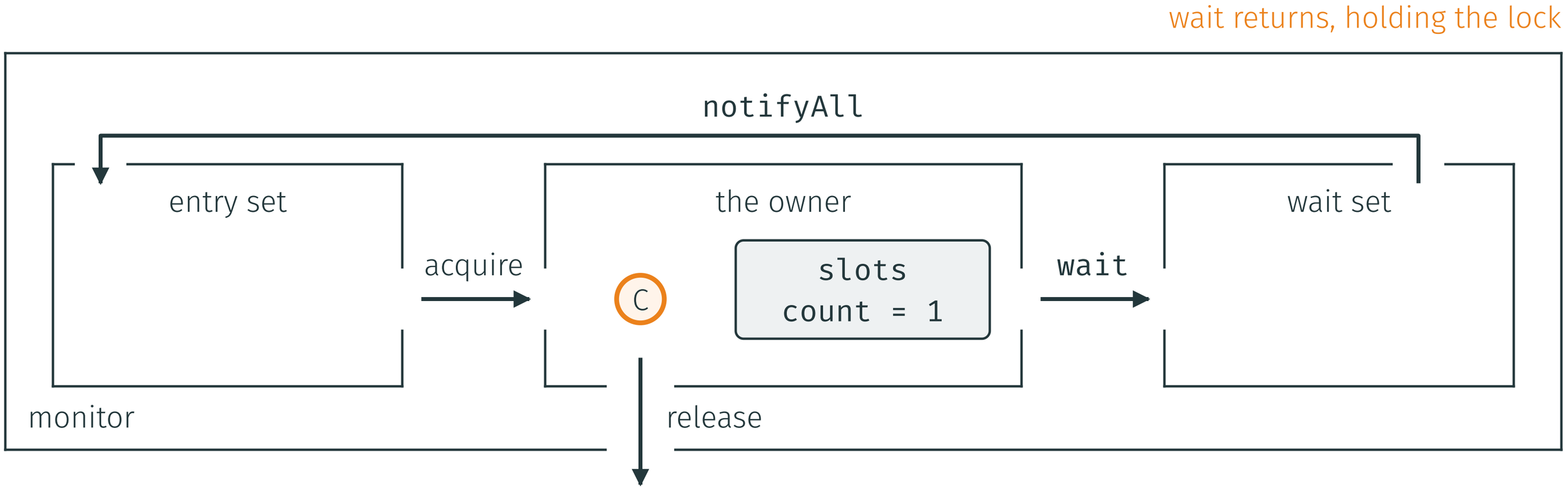
- *number of empty positions*, starting at *N*
- *number of queuing portions*, starting at 0
- *buffer manipulation*, starting at 1

The one that starts at 1 is a **lock**: its P is acquire, its V is release, at most one process between them.

The two that count are **waiting devices**: a P on a counter at zero makes the producer wait for a free place, or the consumer for a portion, until the other's V.

* Dijkstra, Cooperating sequential processes, EWD 123, 1965, section 4.3.

The monitor



Monitor. A module that packages some shared state, one lock that every operation on the state holds, and **condition variables**.*

Java: every object is a monitor; a `synchronized` method holds its lock, `wait()` and `notifyAll()` are its one condition variable. OCaml 5: a record with a `Mutex.t` and one `Condition.t` per thing to wait for.

* Brinch Hansen, Operating System Principles, 1973; Hoare, Communications of the ACM 17(10), 1974.

Example 5: the bounded buffer as a monitor

Java | OCaml 5

Buffer.java ↗

```java
final Integer[] slots = new Integer[K];
int head = 0, tail = 0, count = 0;
public synchronized void put(int x)
    throws InterruptedException {
  while (count == K) wait();   // full
  slots[tail] = x;
  tail = (tail + 1) % K; count++;
  notifyAll();
}
public synchronized Integer take()
    throws InterruptedException {
  while (count == 0) wait();   // empty
  Integer x = slots[head]; slots[head] = null;
  head = (head + 1) % K; count--;
  notifyAll();
  return x;
}
```

A ring of *K* slots and a count. A thread that finds the ring full, or empty, releases the lock and waits, and is woken by the next take, or put.

The invariant: `count` items are in the ring, from `head` on. `put` and `take` keep it, and `wait` is called only while it holds.

```
$ java Buffer while
put:   200000 items, sum 10000100000
taken: sum 10000100000
```

Example 5 with `if`

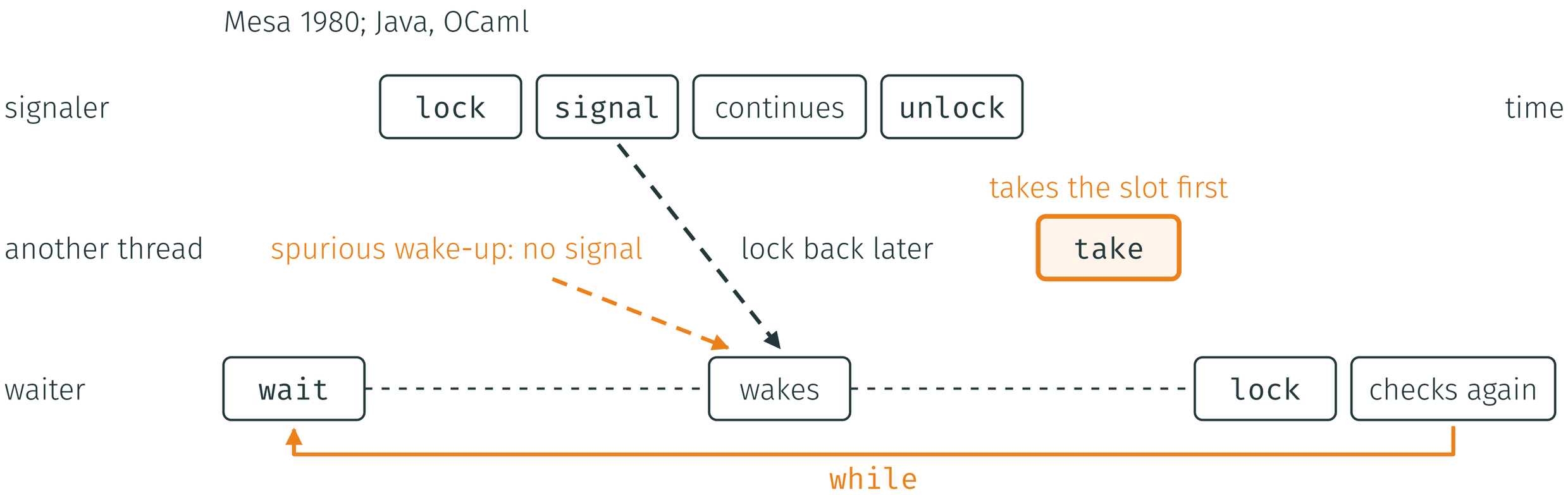
The same buffer with `if (count == 0) wait();` in place of the `while`, and likewise in `put`. Two producers, two consumers. What changes?

1. Nothing: a consumer is woken only when an item has been put.
2. **A consumer woken by a put finds the buffer empty again, the other consumer having taken the item first, and takes from an empty buffer.**
3. Both consumers wait for ever.
4. A consumer takes the same item twice.

```
$ java Buffer if
c1 took from an empty buffer: got null,
   at its take 6 of 100000
```

Who	What	count
C1	take: count is 0, waits, releasing the lock	0
C2	take: count is 0, waits, releasing the lock	0
P	put: stores an item; notifyAll wakes C1 and C2	1
C1	gets the lock first, does not look again, takes the item	0
C2	gets the lock, does not look again, takes from an empty slot	-1

Mesa's monitors and Hoare's



The obligation's fourth part: every wait in a loop that checks its condition again. And `notifyAll` rather than `notify`: an object has one waiting set, and waking one thread may wake one of the wrong kind.